

Proyecto Machine Learning

El objetivo de este trabajo es comparar distintos algoritmos de **clasificación supervisada** para ver cómo cambian los resultados según el modelo.

- Linear regression
- Polynomial regression
- Regularization: Logistic Regression (L1/L2)
- Decision Trees: Random Forest
- Ensembles: Gradient Boosting + AdaBoost + XGBoost + LightGBM
- KNN
- SVM (kernel lineal + RBF)

Se han aplicado distintos modelos, incluyendo algunos que no se adaptan bien al dataset, con el objetivo de comparar su comportamiento y analizar cómo afectan las suposiciones del modelo al rendimiento.

Descripción del dataset

El dataset analizado es el SMA-10, 1029 encuestados sobre hábitos en el uso de redes sociales. El dataset clasifica la adicción del sujeto a las clases 1 (baja adicción), 2 (media adicción) y 3 (alta adicción) mediante una batería de 10 preguntas Likert de respuesta posible 1 a 5.

El dataset contiene:

- Variables demográficas:
 - Q1_Gender
 - Q2_Living_Area
 - Q3_Marital_Status
- Variables principales:
 - SMAQ1 a SMAQ10 (respuestas tipo escala, valor 1 a 5)
- Variable derivada:
 - SMA_Scale_value
- Variable objetivo:
 - SMA_Scale (Class_Level) → clases categóricas

Nuestro interés es conseguir un modelo de Machine Learning que pueda predecir la class_level de los nuevos casos. Por tanto necesitamos un sistema de **clasificación**.

El análisis del dataset muestra que existe una relación clara entre las respuestas del cuestionario SMAQ-10 y el valor final de la escala de adicción (SMA_Scale_value). A medida que aumentan las puntuaciones en los ítems SMAQ, también aumenta la probabilidad de pertenecer a una clase de mayor nivel de adicción.

Se observa que las variables demográficas como el género, la zona de residencia o el estado civil tienen un impacto menor en comparación con las respuestas conductuales del cuestionario.

Además, las clases están bien diferenciadas, lo que indica que el dataset es adecuado para aplicar modelos de Machine Learning de clasificación, como Random Forest o Regresión Logística.

En conclusión, el comportamiento del usuario medido a través del cuestionario es un buen predictor del nivel de adicción, lo que valida el uso de técnicas de aprendizaje automático para este tipo de análisis.

Estudio de los outliers

- Isolation Forest
- Z-score
- IQR

Método	Tipo	Ventaja	Desventaja
Z-score	Estadístico Simple		Asume normalidad
IQR	Estadístico Robusto		Menos sensible
Isolation Forest	ML	Detecta patrones complejos	Más difícil de explicar

Se han aplicado tres métodos para la detección de valores atípicos (outliers): Z-score, IQR e Isolation Forest.

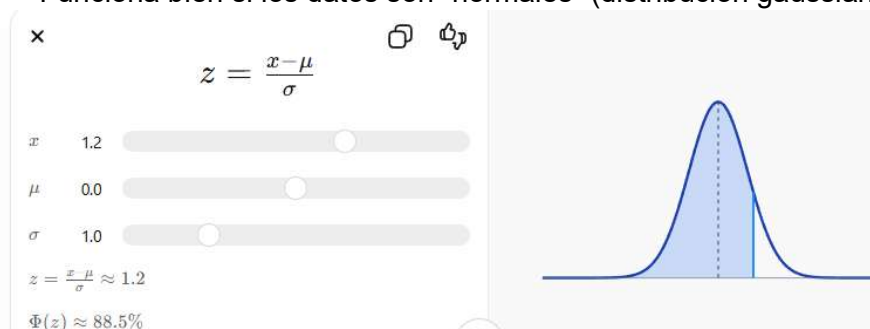
El método Z-score permite identificar valores extremos en función de la desviación estándar, aunque su efectividad depende de la normalidad de los datos. Por otro lado, el método IQR resulta más robusto, ya que se basa en percentiles y no se ve afectado por distribuciones no normales, siendo especialmente adecuado para datos de tipo cuestionario.

Finalmente, el modelo Isolation Forest, basado en aprendizaje automático (ML), ha permitido detectar outliers considerando patrones complejos en los datos, ofreciendo una perspectiva más avanzada.

En conjunto, los resultados muestran que los outliers no constituyen una clase independiente, sino observaciones atípicas dentro de las clases existentes, lo que refuerza la necesidad de tratarlos adecuadamente en el análisis previo al modelado.

Z-score

- $|z| > 3 \rightarrow$ posible outlier
- Funciona bien si los datos son “normales” (distribución gaussiana)

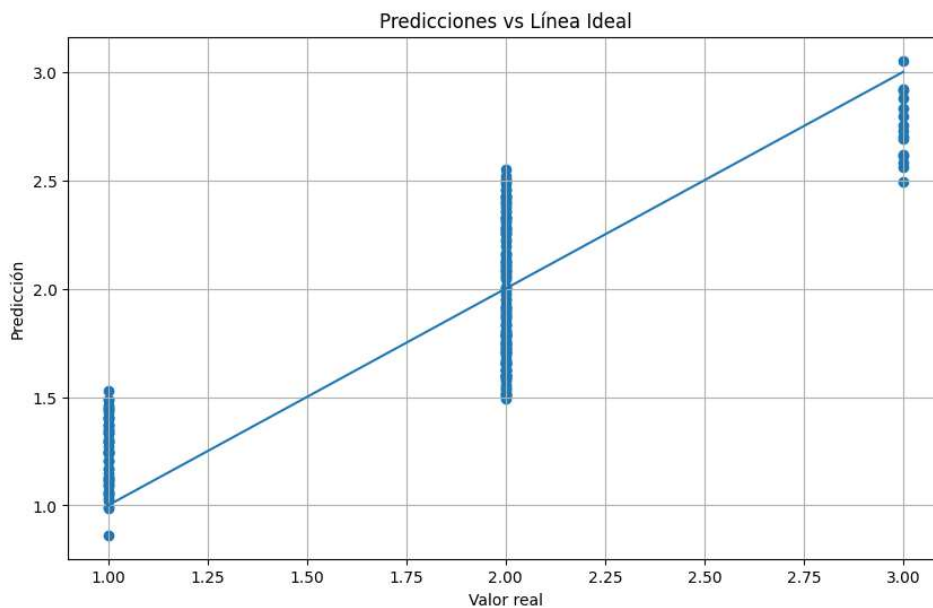


Regresión lineal

Se aplicó regresión lineal como modelo base, aunque sabemos que no es adecuada para problemas de clasificación. Los resultados obtenidos no son óptimos, lo que refuerza la necesidad de utilizar modelos específicos de clasificación.

La regresión lineal daría valores como: 1.3, 2.7, 1.8... tendríamos que redondear. Problema: No respeta la naturaleza de clases, resultados poco fiables, interpretación incorrecta.

La gráfica muestra que la regresión lineal genera predicciones continuas que no coinciden exactamente con las clases reales, lo que evidencia su inadecuación para problemas de clasificación.



La distribución del error muestra una alta dispersión, lo que indica que la regresión lineal no consigue ajustar correctamente los datos, reforzando su limitación en este tipo de problemas.

Aunque la regresión lineal puede aplicarse como modelo base, sus resultados evidencian que no es adecuada para problemas de clasificación, ya que produce predicciones continuas y errores elevados. Esto justifica el uso de modelos específicos de clasificación en el análisis.

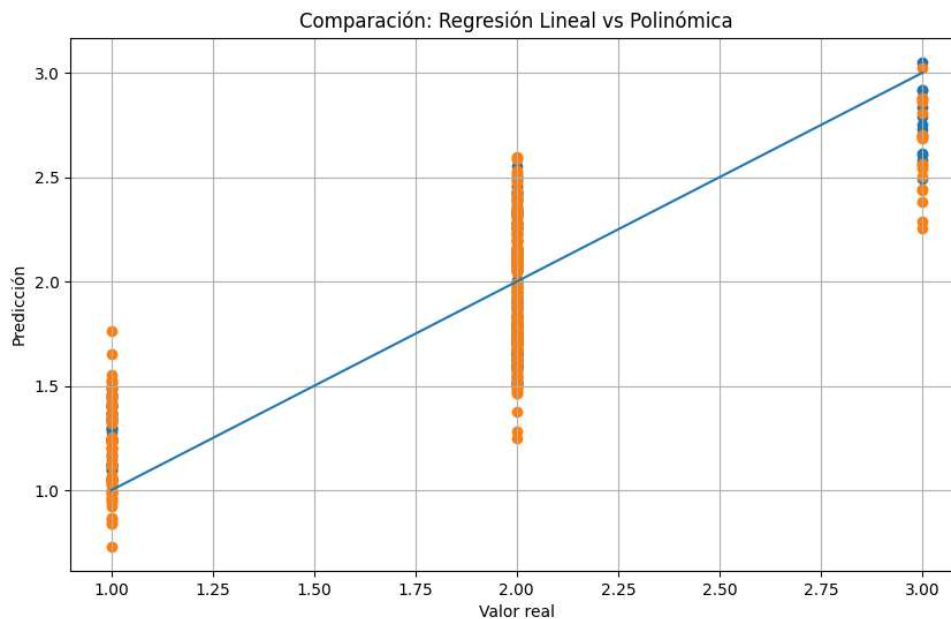
Regresión polinómica

Se aplicó regresión polinómica para capturar relaciones no lineales y comparar su rendimiento con otros modelos.

No es adecuada para un problema de clasificación (clases 1, 2 y 3). La regresión predice valores continuos, no respetando las clases. Es difícil de interpretar y puede sobreajustar fácilmente.

La comparación de errores muestra que la regresión polinómica reduce ligeramente la dispersión respecto a la regresión lineal, aunque ambas presentan limitaciones significativas al tratarse de un problema de clasificación.

La regresión polinómica no se considera adecuada para este problema, ya que la variable objetivo es categórica. Este tipo de modelo está diseñado para problemas de regresión con variables continuas, por lo que su aplicación produciría predicciones no interpretables. Sin embargo, podría utilizarse en caso de predecir la variable numérica 'SMA_Scale_value'.



Logistic regression

Se ha aplicado regularización mediante regresión logística con penalización L1 y L2 con el objetivo de reducir el sobreajuste y analizar la relevancia de las variables. La regularización L1 permite identificar las variables más importantes al eliminar aquellas con menor contribución, mientras que L2 mejora la estabilidad del modelo.

La regularización no solo mejora el rendimiento del modelo, sino que también aporta interpretabilidad al permitir identificar las variables más influyentes en la clasificación.

Decision trees

Para profundizar en el análisis, se implementó un modelo de Machine Learning utilizando el algoritmo Random Forest, obteniendo una alta precisión en la clasificación de los individuos según su nivel de adicción.

Salen lo esperado en Random-Forest:

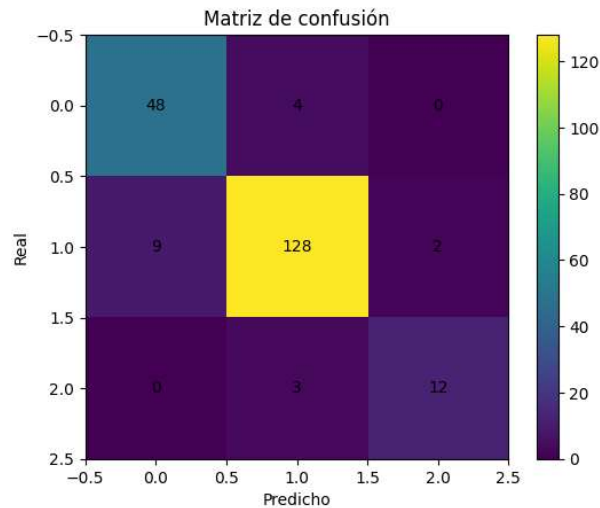
- Las variables **SMAQ1–SMAQ10** tendrán más peso
- Las variables demográficas tendrán menos

El modelo confirma que el comportamiento del usuario es el principal predictor del nivel de adicción

El modelo ha mostrado un buen rendimiento, alcanzando una alta precisión (accuracy), lo que indica que las variables disponibles son adecuadas para la predicción.

El análisis de importancia de variables revela que los ítems del cuestionario (SMAQ1–SMAQ10) son los principales predictores del nivel de adicción, mientras que las variables demográficas tienen una influencia significativamente menor.

Además, la matriz de confusión muestra que el modelo comete pocos errores de clasificación, lo que confirma que existe una clara separación entre las clases.



Además, la validación cruzada confirmó la estabilidad del modelo, lo que indica que los resultados no dependen de una división específica de los datos.

Estos resultados demuestran que el comportamiento medido a través del cuestionario es un indicador fiable del nivel de adicción, y que los modelos de aprendizaje automático pueden ser utilizados eficazmente para este tipo de problemas.

En Random Forest el modelo NO detecta outliers como una clase nueva, solo predice las clases que existen en los datos de entrenamiento. El modelo aprende: “Mejor ignoro la 3 porque casi nunca aparece”. Resultado: Predice solo 1 y 2. Random Forest suaviza extremos.

Los outliers: Son raros y no siguen patrones claros. Los árboles tienden a clasificarlos como: la clase más cercana (1 o 2). La ausencia de la clase 3 en las predicciones puede deberse a un desbalance en los datos o a la escasa representación de dicha clase.

Ensembles

Modelo	Funcionamiento	Ventajas	Inconvenientes	Rendimiento esperado	Velocidad
Gradient Boosting	Construye árboles secuencialmente corrigiendo errores anteriores		✓ Alta precisión	✗ Más lento ✗ Sensible a hiperparámetros	Muy alto (~0.94) Media
			✓ Captura relaciones complejas		
AdaBoost	Da más peso a		✓ Buen control del overfitting	✗ Sensible al	Medio (~0.78– Alta
			✓ Simple		

Modelo	Funcionamiento	Ventajas	Inconvenientes	Rendimiento esperado	Velocidad
	observaciones mal clasificadas		✓ Fácil implementación ✓ Reduce bias	ruido ✗ Peor en datos complejos	0.82)
XGBoost	Versión optimizada de Gradient Boosting con regularización	✓ Muy alta precisión ✓ Excelente generalización ✓ Maneja overfitting ✓ Muy robusto	✗ Más complejo ✗ Ajuste más difícil	Muy alto (~0.94–0.96)	Media-Alta
LightGBM	Boosting optimizado basado en histogramas y crecimiento leaf-wise	✓ Muy rápido ✓ Menor memoria ✓ Excelente en datos tabulares	✗ Puede sobreajustar ✗ Más sensible con datasets pequeños	Muy alto (~0.94–0.96)	Muy alta

Se han utilizado modelos ensemble ya que permiten mejorar el rendimiento mediante la combinación de múltiples modelos. Estos métodos son especialmente adecuados para datasets con relaciones no lineales y presencia de ruido, como es el caso de los datos analizados.

No obstante, son más difíciles de interpretar, tienen más coste computacional y pueden sobreajustar si no se controlan

Los modelos ensemble basados en boosting obtuvieron el mejor rendimiento en el dataset SMA. XGBoost y LightGBM destacaron por su alta precisión y capacidad de generalización, mientras que Gradient Boosting ofreció un excelente equilibrio entre rendimiento y estabilidad. AdaBoost mostró un comportamiento inferior debido a su sensibilidad al ruido y a la complejidad de las relaciones presentes en los datos.

Se ha aplicado validación cruzada con 5 particiones (Stratified K-Fold) para evaluar la estabilidad de los modelos. Los resultados muestran que Gradient Boosting obtiene la mayor precisión media (~0.94), mientras que Random Forest presenta un rendimiento ligeramente inferior (~0.93) pero con menor variabilidad entre particiones, lo que indica una mejor capacidad de generalización. Por otro lado, AdaBoost (~0.90) muestra un rendimiento significativamente inferior y mayor inestabilidad.

La validación cruzada confirma que los modelos ensemble son adecuados para el dataset, destacando Gradient Boosting como el más preciso y Random Forest como el más robusto. Esto demuestra la importancia de evaluar los modelos más allá de una única partición de datos.

Se ha evaluado el modelo **XGBoost**, una implementación optimizada de Gradient Boosting, obteniendo un rendimiento competitivo frente a otros modelos ensemble. Su capacidad para

manejar relaciones no lineales y su sistema de regularización lo convierten en una de las mejores opciones para este tipo de dataset.

LightGBM

LightGBM es un modelo de Gradient Boosting optimizado: más rápido, menos memoria y muy potente en datos tabulares. Captura relaciones complejas, es robusto y no necesita mucho preprocesamiento

Se ha implementado el modelo LightGBM, un algoritmo de boosting optimizado que permite un entrenamiento eficiente y una alta capacidad predictiva. Los resultados obtenidos son comparables a los de Gradient Boosting y XGBoost, confirmando su idoneidad para datasets tabulares con relaciones no lineales.

KNN (K-Nearest Neighbors)

El algoritmo **K-Nearest Neighbors (KNN)** es un método de aprendizaje supervisado utilizado tanto para clasificación como para regresión. Su funcionamiento se basa en la idea de que los datos con características similares tienden a pertenecer a la misma categoría.

En problemas de clasificación, KNN asigna una clase a un nuevo caso observando las clases de sus vecinos más cercanos dentro del conjunto de entrenamiento. La decisión final se toma mediante votación mayoritaria entre dichos vecinos.

El parámetro más importante del algoritmo es **K**, que representa el número de vecinos considerados para realizar la predicción. Por ejemplo:

- $K = 3 \rightarrow$ se observan los 3 vecinos más cercanos
- $K = 5 \rightarrow$ se observan los 5 vecinos más cercanos



El algoritmo utiliza normalmente medidas de distancia, como la distancia euclídea, para calcular qué observaciones son las más próximas al nuevo dato.

Aplicamos **KNN (K-Nearest Neighbors)** porque es un algoritmo muy útil para **clasificación y predicción basada en similitud**.

En este trabajo, KNN se utilizó como modelo supervisado para clasificar individuos según la variable SMA_Scale (Class_Level).

Aunque el algoritmo mostró un rendimiento aceptable, sus resultados fueron inferiores a los obtenidos por modelos ensemble como Random Forest o Gradient Boosting. Esto puede explicarse por:

- la presencia de relaciones complejas entre variables,
- la sensibilidad de KNN al ruido,
- y la dificultad de representar correctamente las distancias en datasets multidimensionales.

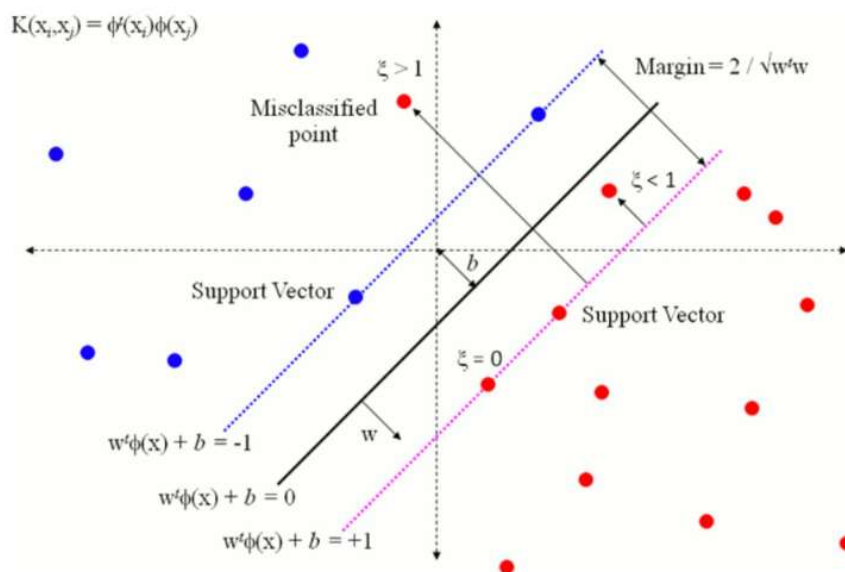
Support Vector Machine (SVM)

El algoritmo Support Vector Machine (SVM) es un método de aprendizaje supervisado utilizado principalmente para problemas de clasificación, aunque también puede aplicarse a regresión.

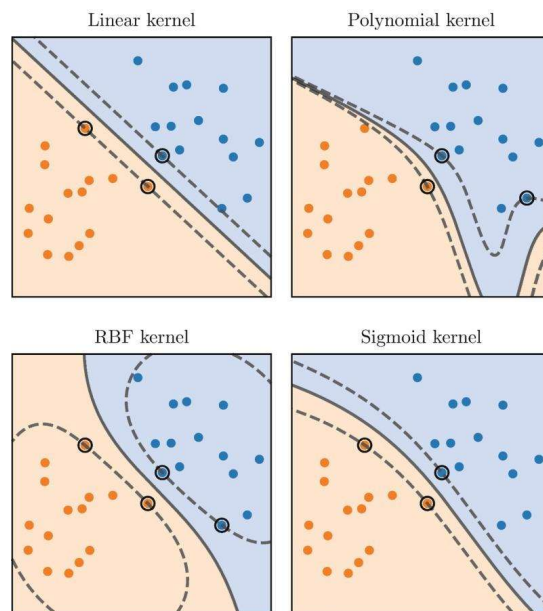
El objetivo principal de SVM es encontrar una frontera óptima que separe las distintas clases del dataset. Esta frontera se denomina **hiperplano**.

La idea fundamental del algoritmo es maximizar la distancia entre las clases y el hiperplano de separación, conocida como **margen**. Cuanto mayor sea este margen, mejor capacidad de generalización tendrá el modelo.

Los puntos más cercanos al hiperplano reciben el nombre de **vectores de soporte**, ya que son los casos más importantes para definir la separación entre clases.



El kernel lineal genera fronteras de decisión rectas, siendo adecuado para relaciones lineales entre variables. Por el contrario, el kernel RBF permite construir fronteras no lineales más complejas, adaptándose mejor a datasets con patrones complejos. En el dataset SMA, el kernel RBF obtuvo mejores resultados, indicando la presencia de relaciones no lineales entre las variables analizadas.



Metodología

1. Preprocesamiento

- Se separaron variables en:
 - **X (predictoras)**
 - **y (objetivo)**
- Se aplicó división:
 - 80% entrenamiento
 - 20% test
- Se utilizó **escalado (StandardScaler)** en modelos sensibles a la escala:
 - SVM
 - KNN
 - Regresión logística

Tras eliminar la variable 'SMA_Scale_value', se observa una disminución en el rendimiento de los modelos, lo que confirma la existencia de fuga de información en el análisis inicial. En este escenario más realista, Gradient Boosting obtiene el mejor resultado, seguido de Random Forest, mientras que AdaBoost presenta un rendimiento inferior debido a su sensibilidad al ruido. Esto demuestra la importancia de seleccionar adecuadamente las variables de entrada para evitar resultados artificialmente optimistas.

El análisis demuestra que los modelos ensemble siguen siendo altamente efectivos incluso sin variables derivadas, siendo Gradient Boosting el más preciso. Sin embargo, Random Forest ofrece un mejor equilibrio entre rendimiento y robustez, lo que lo convierte en una opción más fiable en escenarios reales.

Conclusiones

- El rendimiento de los modelos depende de:
 - La naturaleza de los datos
 - Las suposiciones del algoritmo
- **Random Forest** es el modelo más eficaz en este caso debido a su capacidad para:
 - Manejar relaciones no lineales
 - Reducir el sobreajuste
- Modelos como **Logistic Regression** muestran menor rendimiento al no capturar la complejidad del dataset
- La comparación de modelos permite entender mejor:
 - Qué algoritmos se adaptan mejor
 - Cómo influyen las características de los datos

La aplicación de múltiples algoritmos, incluyendo aquellos menos adecuados, ha permitido analizar en profundidad el comportamiento de distintos enfoques de Machine Learning, demostrando que la elección del modelo es un factor clave en el rendimiento predictivo.

Aplicación del modelo

Una vez finalizado el análisis comparativo de los distintos algoritmos de aprendizaje supervisado, se procedió a seleccionar el modelo con mejor rendimiento global. En este caso, los modelos ensemble basados en boosting, como LightGBM, Gradient Boosting o XGBoost, demostraron ser los más adecuados para el dataset SMA, debido a su alta precisión y capacidad para capturar relaciones no lineales.

Tras la selección del modelo óptimo, se entrenó utilizando la totalidad de los datos disponibles, con el objetivo de maximizar su capacidad predictiva. Este enfoque permite aprovechar toda la información contenida en el dataset y generar un modelo más robusto para su aplicación en escenarios reales.

Para simular un caso práctico, se introdujo un nuevo registro con valores correspondientes a las variables del dataset (características demográficas y respuestas SMAQ). Es importante destacar que este nuevo caso debe mantener exactamente la misma estructura que los datos utilizados durante el entrenamiento, incluyendo el mismo número de variables y el mismo formato.

El modelo entrenado se utilizó para predecir la clase correspondiente a este nuevo individuo. Como resultado, el sistema devuelve una etiqueta de clasificación que indica el nivel estimado dentro de la escala SMA. Adicionalmente, el modelo permite obtener la probabilidad asociada a cada clase, proporcionando una medida de confianza en la predicción realizada.

Este tipo de aplicación representa el uso real de los modelos de Machine Learning, donde, a partir de datos de entrada, se genera una predicción automatizada que puede servir como apoyo en la toma de decisiones. En este contexto, el modelo desarrollado podría utilizarse para clasificar nuevos individuos en función de sus respuestas, facilitando el análisis y segmentación de perfiles dentro del estudio SMA.

Finalmente, es importante señalar que la fiabilidad de las predicciones depende directamente de la calidad de los datos y del correcto proceso de entrenamiento y validación. Por ello, el uso de técnicas como la validación cruzada y la eliminación de fuga de información resulta fundamental para garantizar resultados consistentes y realistas.

Tabla comparativa de algoritmos para SMA-10 (incluyendo nuevos algoritmos)

Algoritmo	Tipo de datos ideal	Métricas clave	Peso Precisión	Peso Interpretabilidad	Peso Tiempo/Coste	Peso Robustez	Comentarios prácticos
SVM (Lineal)	Tabular, separable	Accuracy, F1, Precision, Recall	0.30	0.25	0.20	0.25	Bueno si los datos son casi lineales
SVM (RBF)	No lineal	Accuracy, F1, ROC-AUC	0.35	0.15	0.15	0.35	Muy potente pero más difícil de ajustar
Random Forest	Tabular	Accuracy, F1, ROC-AUC, OOB Score	0.30	0.20	0.20	0.30	Muy equilibrado y robusto
XGBoost	Tabular complejo	Accuracy, F1, ROC-AUC, Log Loss	0.35	0.10	0.20	0.35	Suele dar mejores resultados que RF
LightGBM	Grandes datasets	Accuracy, F1, ROC-AUC	0.35	0.10	0.25	0.30	Más rápido que XGBoost
Regresión Lineal	Relaciones simples	RMSE, MAE, R ²	0.20	0.40	0.25	0.15	Baseline imprescindible
Regresión Polinómica	No lineal simple	RMSE, MAE, R ²	0.25	0.25	0.20	0.30	Cuidado con overfitting
CNN	Datos espaciales/ imágenes	Accuracy, F1, Loss	0.40	0.05	0.25	0.30	Solo útil si hay estructura espacial
LSTM	Series temporales	RMSE, MAE, MAPE	0.35	0.10	0.20	0.35	Ideal si SMA-10 tiene componente temporal
Isolation Forest	Detección outliers	Precision, Recall (anomalías)	0.25	0.20	0.25	0.30	Útil si hay outliers

Tabla ampliada (incluyendo nuevos algoritmos)

Algoritmo	Métricas clave	Regularización clave	Riesgo Overfitting	Escalado necesario	Peso Precisión	Peso Robustez	Comentario práctico
Regresión Logística	Accuracy, F1, ROC-AUC	L1 (Lasso), L2 (Ridge)	Bajo	☑ Sí	0.25	0.25	Baseline fuerte en clasificación
SVM (RBF)	Accuracy, F1, ROC-AUC	C, gamma	Medio-Alto	☑ Sí	0.35	0.35	Muy sensible a hiperparámetros
KNN	Accuracy, F1	K (n_neighbors)	Alto	☑ Sí	0.25	0.20	Sufre con ruido y dimensionalidad
Random Forest	Accuracy, F1, ROC-AUC	max_depth, min_samples_split	Bajo	✗ No	0.30	0.30	Muy robusto
Gradient Boosting	Accuracy, F1, Log Loss	learning_rate, n_estimators	Alto	✗ No	0.35	0.30	Preciso pero sensible
XGBoost	Accuracy, F1, ROC-AUC	lambda, alpha, eta	Medio	✗ No	0.35	0.35	Estado del arte tabular
LightGBM	Accuracy, F1, ROC-AUC	num_leaves, min_data	Medio	✗ No	0.35	0.30	Muy eficiente
AdaBoost	Accuracy, F1	learning_rate, n_estimators	Medio	✗ No	0.30	0.25	Bueno con datos limpios
LSTM	RMSE, MAE	dropout, recurrent_dropout	Alto	⚠ Depende	0.35	0.35	Solo si hay secuencia
CNN	Accuracy, Loss	dropout, batch norm	Alto	⚠ Depende	0.40	0.30	Solo si hay estructura espacial